

PROJEKLEITFADEN

Quiz-Webanwendung

Von der ersten Idee bis zur Abnahme

Teamgröße: **3 Entwickler**

Projektdauer: **8 Projekttag**

Stack: **HTML, CSS, JavaScript, PHP, MySQL/MariaDB**

Zielumgebung: **Lokales Kursraum-Netzwerk, nur PC/Laptop (kein WLAN, keine Smartphones)**

So liest du dieses Dokument

Dieser Leitfaden ist eine Schritt-für-Schritt-Anleitung. Er ist bewusst für jemanden geschrieben, der noch nie ein Webprojekt im Team gemacht hat. Lies ihn einmal komplett durch, bevor ihr anfangt – auch die Teile, die nicht dich betreffen. Sonst weißt du nicht, was deine Teamkollegen brauchen und wo eure Schnittstellen sind.

Das Dokument folgt der gleichen Reihenfolge, in der ihr im Projekt vorgehen sollt:

1. Phase 0 – Vorbereitung (vor Tag 1): Tools, Repository, Server, Spielregeln im Team.
2. Phase 1 – Gemeinsame Planung (Tag 1 Vormittag): Datenbank, API, UI, Aufgabenverteilung.
3. Phase 2 – Umsetzung (Tag 1 Nachmittag bis Tag 7): Jeder baut seinen Bereich, ihr integriert täglich.
4. Phase 3 – Integration, Testing, Abnahme (Tag 7 bis Tag 8): Alles zusammenführen, Bugfixing, Präsentation.

Wichtigste Regel

Schreibt am Tag 1 keine Zeile Code, bevor das Datenbankschema und die API-Endpoints schriftlich festgelegt sind. Das ist der häufigste Fehler in 8-Tage-Projekten. Wenn drei Leute parallel ohne abgestimmte Schnittstellen losbauen, verbringt ihr Tag 5 und 6 mit Refactoring statt mit Features.

1. Was bauen wir eigentlich?

Bevor wir über Code reden: Stell dir vor, wie das Endprodukt im Unterricht eingesetzt wird. Diese Geschichte ist wichtiger als jede Spezifikation.

1.1 Die Geschichte des Endprodukts

Der Dozent öffnet im Browser die Quiz-Anwendung am Lehrer-PC, klickt auf „Hosten“, wählt den Pool „Python Grundlagen“, stellt 10 Fragen mit 30 Sekunden Zeitlimit und Punkte-Modus „Teilpunkte“ ein. Das System zeigt einen Teilnahme-Code, z. B. A7K9, gut sichtbar am Beamer.

Die Teilnehmer öffnen an ihren Kurs-PCs/Laptops eine URL im lokalen Netzwerk, geben den Code ein, tippen einen Nicknamen, landen in der Lobby. Der Dozent sieht die Liste der Beigetretenen, startet das Spiel. Frage 1 erscheint groß am Beamer, gleichzeitig in den Browsern aller Teilnehmer mit den vier Antworten als großen Buttons. Die Teilnehmer wählen aus, klicken „Antworten einloggen“. Wer schneller ist, bekommt mehr Punkte. Nach 30 Sekunden oder wenn alle eingeloggt haben, schließt der Server die Frage, zeigt die Lösung, dann den Zwischenstand. Der Dozent erklärt kurz, klickt „Nächste Frage“. So geht es zehn Mal. Am Ende: Endranking, Applaus, Ergebnis gespeichert.

Das ist der Kern. Alles andere – Admin-Bereich, CSV-Import, Bilder, Registrierung – existiert nur, damit dieser Kern funktioniert.

1.2 Vier Rollen mit unterschiedlichen Rechten

Die Anwendung kennt vier Rollen. Wichtig: Nur drei davon spielen aktiv mit. Die vierte (Admin) ist reine Verwaltung.

Rolle	Darf hosten?	Darf beitreten?	Sieht Statistik?	Verwaltungsrechte
Gast	Ja	Ja	Nein – Session läuft temporär	Keine
User	Ja	Ja	Eigene Historie pro Thema	Keine
Dozent	Ja	Ja	Eigene Historie + Themen-Übersicht (falsch beantwortete Fragen)	Erstellt Fragenpools, Fragen, Bilder; Excel-/CSV-Import
Admin	Nein	Nein	Themen-Übersicht (wie Dozent)	Alles vom Dozenten plus: Benutzerverwaltung (anlegen, bearbeiten, löschen, Passwort zurücksetzen)

Diese Tabelle ist die wichtigste Übersicht des gesamten Projekts. Hängt sie als Spickzettel an die Wand. Bei jedem Endpoint und jeder UI-Seite müsst ihr fragen: Welche Rolle darf das?

Wichtige Konsequenzen

- Gast-Daten sind komplett temporär. Während des Spiels werden sie technisch in der Datenbank gespeichert (sonst funktioniert die Spiellogik nicht), aber sie werden nicht für persönliche Statistiken oder Historie verwendet.
- User sehen nur ihre eigene Historie: bei welchem Thema haben sie wie abgeschnitten.
- Dozent und Admin sehen zusätzlich eine themenübergreifende Auswertung: welche Fragen wurden am häufigsten falsch beantwortet – das hilft im Unterricht zu erkennen, wo Bedarf besteht.
- Pro Thema gibt es genau einen Fragenpool. Dozent (und Admin) erstellen diese Pools und befüllen sie – entweder manuell am Bildschirm oder per Excel-/CSV-Import.
- Der Admin ist ein reiner Verwalter. Er hostet nicht, spielt nicht, tritt nicht bei. Er hat aber alle inhaltlichen Rechte des Dozenten plus exklusiv die Benutzerverwaltung.

Wichtige Abweichung vom Lastenheft

Im Kursraum steht kein WLAN zur Verfügung, deshalb wird die Anwendung ausschließlich für PC/Laptop entwickelt. Das Lastenheft (Kap. 4.4, 12.3) fordert eigentlich auch Smartphone-Tauglichkeit – diese Anforderung entfällt in eurer Umsetzung. Zusätzlich habt ihr das Rollenkonzept gegenüber Kap. 5 erweitert: vier Rollen statt drei, mit der neuen Trennung Admin (Verwaltung) und Dozent (Inhalte plus Hosting). Notiert beide Abweichungen schriftlich in eurer Projektdokumentation und stimmt sie mit dem Dozenten ab.

2. Phase 0 – Vorbereitung (vor Tag 1)

Diese Phase passiert idealerweise einen Tag vor Projektstart oder am Morgen von Tag 1, bevor das Lastenheft durchgesprochen wird. Sie kostet ca. 2 Stunden und spart euch später Tage.

2.1 Entwicklungsumgebung einrichten

Jeder im Team braucht dasselbe Setup. Sonst sagt einer „läuft bei mir“ und alle anderen verlieren Zeit.

Empfehlung für alle drei Entwickler

- XAMPP (Windows/Linux/Mac) oder Laragon (Windows) installieren. Beide bringen Apache, PHP 8.x und MariaDB mit. Laragon ist auf Windows komfortabler, XAMPP plattformneutral.
- Visual Studio Code als Editor, mit den Extensions: PHP Intelephense, ESLint, Prettier, Live Server.
- Git installieren und Konten auf GitHub anlegen, falls nicht vorhanden.
- Ein moderner Browser mit guten DevTools (Chrome oder Firefox).
- Optional, aber empfohlen: TablePlus, DBeaver oder phpMyAdmin (kommt mit XAMPP) zum Inspizieren der Datenbank.

Tipp zum lokalen Server

Stellt euren lokalen Server so ein, dass alle PHP-Fehler angezeigt werden – während der Entwicklung, nicht in Produktion. In der `php.ini`: `display_errors=On`, `error_reporting=E_ALL`. Sonst tippt ihr im Dunkeln, wenn etwas schiefgeht.

2.2 Git-Repository einrichten

Einer von euch (am besten der mit der meisten Git-Erfahrung) legt das Repository an. Die anderen klonen es.

5. Repository auf GitHub erstellen, z. B. `quiz-webapp`, mit einer leeren README und einer `.gitignore` für PHP.
6. Repository lokal klonen: `git clone <url>`.
7. Sofort eine Ordnerstruktur anlegen (siehe Kapitel 6) und einen ersten Commit machen.
8. Alle drei pushen einen kleinen Testcommit, damit klar ist: Jeder kann pushen und pullen.

Branching-Strategie für 3 Personen / 8 Tage

Macht es einfach. Komplexe Git-Flows brauchen wir hier nicht.

- `main` – ist immer lauffähig. Niemand pusht direkt auf `main`.
- `dev` – der Sammel-Branch. Hier integriert ihr eure Features täglich.
- `feature/<name>` – pro Feature ein Branch, z. B. `feature/lobby-host`, `feature/csv-import`.

Workflow pro Feature:

9. Von `dev` abzweigen: `git checkout dev && git pull && git checkout -b feature/lobby-host`.

10. Arbeiten, kleine Commits machen mit verständlichen Nachrichten.
11. Wenn fertig: push und Pull Request auf dev erstellen.
12. Ein anderes Teammitglied schaut kurz drüber, dann mergen.

Merge-Konflikte vermeiden

Pullt mindestens zweimal am Tag den dev-Branch in euren Feature-Branch. Je länger ihr wartet, desto wilder werden die Konflikte. Eine 5-Minuten-Pull-Routine erspart 2-Stunden-Konflikt-Sessions.

2.3 Kommunikation und Spielregeln

Drei Leute, acht Tage, ein Projekt. Klare Absprachen sind kein Bürokratie-Kram, sondern überlebenswichtig.

Was	Wann	Dauer	Inhalt
Daily Standup	Jeden Morgen 09:00	15 Min	Was habe ich gestern gemacht, was heute, wo blockiere ich?
Integrations-Check	Jeden Tag 16:00	30 Min	dev-Branch zusammenführen, gemeinsam durchklicken
Wochenmitte-Review	Tag 4 Nachmittag	60 Min	Stand vs. Plan, was muss raus, was bleibt?

Kommunikationskanal: Eine WhatsApp-/Signal-/Discord-Gruppe für schnelle Fragen, der Rest läuft über GitHub Issues und Pull-Request-Kommentare.

2.4 Mehrere Teilnehmer ohne WLAN simulieren

Da im Kursraum kein WLAN zur Verfügung steht und ihr trotzdem mit mehreren Teilnehmern testen müsst, gibt es drei Strategien. Alle drei sind nützlich – ihr werdet alle drei einsetzen.

Strategie 1 – Mehrere Browser-Fenster auf einem PC (für tägliche Entwicklung)

Jeder Entwickler kann allein einen Mehrspieler-Test machen, indem er mehrere isolierte Browser-Sessions öffnet. Jedes Fenster wird zu einem eigenen Teilnehmer.

- Fenster 1: Normaler Chrome → Host
- Fenster 2: Chrome Inkognito → Teilnehmer 1
- Fenster 3: Firefox → Teilnehmer 2
- Fenster 4: Firefox Private → Teilnehmer 3
- Alternativ: Chrome-Profil (Schaltfläche oben rechts → Profil hinzufügen). Damit könnt ihr beliebig viele isolierte Sessions öffnen.

Wichtig: Cookies/Sessions müssen getrennt sein, sonst verwechselt der Server die Teilnehmer. Inkognito und Profile sorgen für getrennte Cookies. Zwei normale Chrome-Fenster teilen sich die Session – das funktioniert nicht.

Strategie 2 – Mehrere PCs im lokalen Netzwerk (für Integrationstests)

Sobald ihr drei oder vier PCs zur Verfügung habt (z. B. eure eigenen Notebooks plus die Kurs-Rechner), spielt ihr von echten Geräten gegeneinander. Setup:

13. Ein PC wird zum Server. Apache/PHP läuft dort. IP-Adresse mit ipconfig (Windows) oder ip a (Linux) ermitteln, z. B. 192.168.1.42
14. Apache so konfigurieren, dass er auf allen Interfaces lauscht (Listen 0.0.0.0:80 in httpd.conf)
15. Windows-Firewall des Server-PCs für Port 80 freigeben (eingehende Regel)
16. Die anderen PCs öffnen im Browser <http://192.168.1.42/> – das ist eure Quiz-Anwendung
17. Test: alle PCs müssen den Server-PC anpingen können (ping 192.168.1.42)

Strategie 3 – Generalprobe in echter Umgebung

Spätestens am Tag 7 Abend oder Tag 8 Vormittag müsst ihr in den realen Kursraum mit den realen Rechnern. Klärt vorab: Sind die Kurs-PCs alle im gleichen LAN? Welche IPs haben sie? Darf einer eurer Laptops als Server ins Kurs-LAN? Sind Firewalls eingeschränkt? Wenn ihr das erst am Präsentationstag merkt, ist es zu spät.

Praktischer Tipp für die Entwicklung

Während der Entwicklung reicht Strategie 1 (mehrere Browser-Fenster auf einem PC) für 90 % aller Tests. Greift erst dann zu Strategie 2, wenn ihr netzwerkspezifische Bugs vermutet – z. B. wenn die Polling-Last erst bei echten 5 Browsern auf 5 Rechnern problematisch wird. Verschwendet keine Zeit auf umständliches LAN-Setup, solange ihr lokal sauber testen könnt.

3. Tag 1 Vormittag – Gemeinsame Planung

Am Tag 1 sitzt ihr zu dritt an einem Tisch (oder im Call) mit dem Lastenheft, einem Whiteboard und Kaffee. Diese 3–4 Stunden sind die wichtigsten des gesamten Projekts.

3.1 Lastenheft gemeinsam durchsprechen (60 Min)

Lest die Punkte 1 bis 33 des Lastenhefts laut durch. Bei jedem Punkt fragt ihr: „Verstehen wir das gleich?“ Was unklar ist, notiert ihr auf einer Liste „Offene Fragen“. Diese Liste klärt ihr am Ende des Vormittags mit dem Dozenten.

Markiert dabei jeden Punkt:

- MUSS – aus Kapitel 29 des Lastenhefts. Diese müssen am Ende funktionieren.
- SOLL – aus Kapitel 30. Schön, aber verzichtbar, wenn die Zeit knapp wird.
- KÜR – aus Kapitel 31. Nur, wenn alles andere fertig und stabil ist.

3.2 Datenbankmodell entwerfen (60 Min)

Setzt euch ans Whiteboard und zeichnet die Tabellen. Diese müssen sitzen, bevor irgendjemand Code schreibt. Eine fertige Vorlage findet ihr in Kapitel 4.

Vorgehen:

18. Listet die Hauptobjekte: users, question_pools, questions, answers, games, game_participants, game_questions, participant_answers.
19. Für jede Tabelle: Welche Spalten? Welcher Primärschlüssel? Welche Fremdschlüssel?
20. Beziehungen klären: Ein Spiel hat viele Teilnehmer (1:n), ein Pool hat viele Fragen (1:n) usw.
21. Schreibt das Schema als SQL-Skript auf (init.sql). Das ist euer Vertrag.

3.3 API-Endpoints festlegen (45 Min)

Ohne klare API kann das Frontend nicht arbeiten, bevor das Backend fertig ist. Definiert deshalb am Tag 1 alle Endpoints und deren Request-/Response-Format. Eine Vorlage liegt in Kapitel 5.

Pro Endpoint braucht ihr:

- URL (z. B. /api/game/create.php)
- HTTP-Methode (GET, POST)
- Eingaben (JSON-Body oder Query-Parameter)
- Ausgaben (JSON-Struktur bei Erfolg, bei Fehler)
- Wer darf das aufrufen? (Gast, Host mit Token, Admin)

3.4 UI-Skizzen (30 Min)

Skizziert auf Papier oder im Whiteboard die wichtigsten Screens:

#	Screen	Wichtigster Inhalt
1	Startseite	Zwei große Buttons: Teilnehmen und Hosten. Login-Link für angemeldete Nutzer.
2	Login	E-Mail + Passwort. Nach Login: Weiterleitung auf Dashboard je nach Rolle.
3	Beitreten	Code-Eingabe, Nickname, optional Avatar
4	Lobby Teilnehmer	Teilnehmerliste, Wartetext
5	Lobby Host	Code groß, Teilnehmerliste, Einstellungen, Start-Button
6	Frage (Teilnehmer)	Frage groß, 4 Antworten im 2x2-Raster, Timer, Speichern-Button
7	Frage (Host/Beamer)	Frage groß, Antworten, Timer, Status x von y geantwortet
8	Lösung + Zwischenstand	Richtige Antwort markiert, optional Erklärung, Punktetabelle
9	Endranking	Top-Liste mit Platz, Name, Punkten, korrektem Gleichstand
10	User-Dashboard / Historie	Eigene Statistik pro Thema: Spiele, Punkte, Trefferquote
11	Dozent-Dashboard	Themen-Übersicht: welche Fragen wurden am häufigsten falsch beantwortet
12	Pool- und Fragen-Verwaltung	Liste der Pools, Liste der Fragen, Formular zum Anlegen/Bearbeiten, Excel-/CSV-Import
13	Benutzerverwaltung (nur Admin)	User-Liste, Anlegen, Bearbeiten, Passwort zurücksetzen, Löschen

3.5 Aufgabenteilung festlegen (30 Min)

Siehe Kapitel 7 für einen konkreten Vorschlag. Wichtig ist: Jeder hat einen Hauptbereich und ist dafür verantwortlich. Ihr helft euch trotzdem gegenseitig, aber Verantwortung muss klar sein.

4. Datenbank-Design

Das ist ein konkreter Vorschlag, der alle Anforderungen des Lastenhefts abdeckt. Passt es an, wenn ihr bessere Ideen habt, aber haltet euch grundsätzlich an diese Struktur.

4.1 Tabellen-Übersicht

Tabelle	Zweck
users	Registrierte Nutzer mit 3 Rollen: user, dozent, admin. Lastenheft Kap. 22 + erweitertes Rollenkonzept
question_pools	Fragenpools (Themen). Lastenheft Kap. 23.2
questions	Einzelne Fragen mit den 4 Antworten. Lastenheft Kap. 23.3
media	Hochgeladene Bilder. Lastenheft Kap. 24
games	Konkrete Spielinstanzen mit Code, Host-Token, Einstellungen, Status. Lastenheft Kap. 9
game_participants	Teilnehmer in einem Spiel (Gast oder User-Verknüpfung)
game_questions	Reihenfolge der Fragen und gemischte Antwortreihenfolge pro Spiel (Lastenheft Kap. 11.4)
participant_answers	Die abgegebene Antwort eines Teilnehmers auf eine Frage des Spiels
game_results	Endergebnisse für die Historie. Lastenheft Kap. 21

4.2 SQL-Schema als Startpunkt

Das ist nicht das fertige Schema. Es ist ein Startpunkt, den ihr am Tag 1 zusammen verfeinert. Achtet auf die Kommentare – sie zeigen, wo es Entscheidungen zu treffen gibt.

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,          -- password_hash() Ergebnis
  username VARCHAR(50) NOT NULL,
  role ENUM('user', 'dozent', 'admin') NOT NULL DEFAULT 'user',
  avatar VARCHAR(50) NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE question_pools (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,             -- pro Thema genau ein Pool
  description TEXT NULL,
  created_by INT NULL,                           -- user_id (Dozent oder Admin)
  is_active TINYINT(1) NOT NULL DEFAULT 1,
```

```

    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE media (
    id INT AUTO_INCREMENT PRIMARY KEY,
    filename VARCHAR(255) NOT NULL,           -- intern erzeugter Name
    original_name VARCHAR(255) NOT NULL,
    mime_type VARCHAR(50) NOT NULL,
    size_bytes INT NOT NULL,
    uploaded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    uploaded_by INT NULL,
    FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE questions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pool_id INT NOT NULL,
    question_text TEXT NOT NULL,
    image_id INT NULL,
    answer_a TEXT NOT NULL,
    answer_b TEXT NOT NULL,
    answer_c TEXT NOT NULL,
    answer_d TEXT NOT NULL,
    correct_a TINYINT(1) NOT NULL DEFAULT 0,  -- 1 wenn A richtig
    correct_b TINYINT(1) NOT NULL DEFAULT 0,
    correct_c TINYINT(1) NOT NULL DEFAULT 0,
    correct_d TINYINT(1) NOT NULL DEFAULT 0,
    explanation TEXT NULL,
    is_active TINYINT(1) NOT NULL DEFAULT 1,
    FOREIGN KEY (pool_id) REFERENCES question_pools(id),
    FOREIGN KEY (image_id) REFERENCES media(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE games (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pool_id INT NOT NULL,
    join_code VARCHAR(6) NOT NULL UNIQUE,      -- 4-6 Zeichen, A-Z 2-9
    host_token VARCHAR(64) NOT NULL UNIQUE,    -- geheim, z.B.
    bin2hex(random_bytes(32))
    host_user_id INT NULL,                     -- NULL wenn Gast-Host
    host_plays TINYINT(1) NOT NULL DEFAULT 0,
    num_questions INT NOT NULL,
    time_limit_sec INT NOT NULL,

```

```

    score_mode ENUM('all_or_nothing','partial') NOT NULL,
    status ENUM('lobby','running','finished','aborted','archived') NOT NULL
DEFAULT 'lobby',
    current_question_index INT NOT NULL DEFAULT 0,  -- 0-basiert
    current_question_started_at DATETIME NULL,      -- für Server-Timer
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    finished_at DATETIME NULL,
    FOREIGN KEY (pool_id) REFERENCES question_pools(id),
    FOREIGN KEY (host_user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE game_participants (
    id INT AUTO_INCREMENT PRIMARY KEY,
    game_id INT NOT NULL,
    user_id INT NULL,                -- NULL = Gast
    display_name VARCHAR(50) NOT NULL,
    avatar VARCHAR(50) NULL,
    session_token VARCHAR(64) NOT NULL,    -- Teilnehmer-Auth
    is_host_player TINYINT(1) NOT NULL DEFAULT 0,
    removed TINYINT(1) NOT NULL DEFAULT 0,  -- vom Host entfernt
    joined_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE game_questions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    game_id INT NOT NULL,
    question_id INT NOT NULL,
    position INT NOT NULL,          -- 0..n-1
    shuffle_order VARCHAR(20) NOT NULL,  -- z.B. 'C,A,D,B' = gemischte
Reihenfolge
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
    FOREIGN KEY (question_id) REFERENCES questions(id),
    UNIQUE KEY (game_id, position)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE participant_answers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    participant_id INT NOT NULL,
    game_question_id INT NOT NULL,
    selected_a TINYINT(1) NOT NULL DEFAULT 0,
    selected_b TINYINT(1) NOT NULL DEFAULT 0,
    selected_c TINYINT(1) NOT NULL DEFAULT 0,

```

```

selected_d TINYINT(1) NOT NULL DEFAULT 0,
points INT NOT NULL DEFAULT 0,
is_fully_correct TINYINT(1) NOT NULL DEFAULT 0,
answered_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
committed TINYINT(1) NOT NULL DEFAULT 0, -- explizit eingeloggt
FOREIGN KEY (participant_id) REFERENCES game_participants(id) ON DELETE
CASCADE,
FOREIGN KEY (game_question_id) REFERENCES game_questions(id) ON DELETE
CASCADE,
UNIQUE KEY (participant_id, game_question_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE game_results (
  id INT AUTO_INCREMENT PRIMARY KEY,
  game_id INT NOT NULL,
  participant_id INT NOT NULL,
  user_id INT NULL, -- für Historie registrierter Nutzer
  pool_id INT NOT NULL,
  display_name VARCHAR(50) NOT NULL,
  total_points INT NOT NULL,
  fully_correct_count INT NOT NULL,
  rank_position INT NOT NULL,
  participants_count INT NOT NULL,
  finished_at DATETIME NOT NULL,
  FOREIGN KEY (game_id) REFERENCES games(id),
  FOREIGN KEY (participant_id) REFERENCES game_participants(id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
  FOREIGN KEY (pool_id) REFERENCES question_pools(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Warum die Antwortreihenfolge in game_questions gespeichert wird

Das Lastenheft Kap. 11.4 verlangt, dass die gemischte Reihenfolge pro Spiel fix bleibt und für alle Teilnehmer gleich ist. Wenn ihr beim Anzeigen jedes Mal neu mischt, sieht jeder Teilnehmer eine andere Reihenfolge und nach Reload steht alles woanders. Das Mischen passiert einmal beim Spielstart und wird in shuffle_order persistiert.

4.3 Umgang mit Gast-Daten

Eure Anforderung: „Gast-Sessions sind komplett temporär, keine Daten gespeichert.“ Das lässt sich nicht hundertprozentig wörtlich umsetzen, weil das Spiel ohne Persistenz der Antworten technisch nicht funktioniert. Hier die pragmatische Lösung:

- Während des Spiels: Antworten aller Teilnehmer (auch Gäste) werden in participant_answers gespeichert. Ohne das funktioniert die Punkteauswertung nicht.

- Nach Spielende: game_results wird nur für registrierte User geschrieben (user_id IS NOT NULL). Gäste erscheinen im aktuellen Endranking, aber nicht in einer dauerhaften persönlichen Historie.
- Für die Themen-Statistik des Dozenten: aggregiert über alle participant_answers, anonym. Der Dozent sieht „diese Frage wurde 23-mal gestellt, davon 14-mal falsch beantwortet“ – ohne Namen, ohne Zuordnung zu Gästen oder Usern.
- Bei Spielarchivierung: Personenbezogene Daten von Gast-Teilnehmern (display_name) können auf einen Platzhalter wie „Gast“ gesetzt werden, während die Antwortdaten für aggregierte Statistik erhalten bleiben.

Datenschutz-Gedanke

Diese Lösung respektiert die Anforderung im Geist: Gäste haben keine persönliche Identität in der Datenbank über das Spiel hinaus, sehen keine eigene Historie und können nicht nachverfolgt werden. Trotzdem fließen ihre anonymen Antworten in die Themen-Statistik – das ist wertvoll, weil sonst Spiele mit überwiegend Gästen für die Dozenten-Auswertung wertlos wären.

5. API-Endpoints

Alle Endpoints geben JSON zurück und erwarten JSON oder Form-Daten als Input. Format der Antwort einheitlich:

```
// Erfolg: { "ok": true, "data": {...} }
// Fehler: { "ok": false, "error": "Verständliche Fehlermeldung" }
```

5.1 Öffentliche Endpoints (Gäste)

Methode	URL	Zweck
POST	/api/game/join.php	Spiel beitreten mit join_code + display_name
POST	/api/participant/answer.php	Antwort speichern (nicht committed) für laufende Frage
POST	/api/participant/commit.php	Antwort final einloggen
GET	/api/game/state.php	Spielstatus pollen: Lobby? Welche Frage? Restzeit? Lösung?
POST	/api/auth/login.php	Login mit E-Mail + Passwort
POST	/api/auth/register.php	Registrierung (SOLL, optional)
POST	/api/auth/logout.php	Abmelden

5.2 Host-Endpoints (host_token erforderlich)

Methode	URL	Zweck
POST	/api/host/create.php	Neues Spiel erstellen, gibt join_code + host_token zurück
GET	/api/host/lobby.php	Teilnehmerliste + Einstellungen abrufen
POST	/api/host/start.php	Spiel starten, Fragen zufällig ziehen, Antworten mischen
POST	/api/host/next.php	Zur nächsten Frage wechseln
POST	/api/host/reveal.php	Lösung freigeben (optional, oft automatisch nach Timer)
POST	/api/host/kick.php	Teilnehmer aus Spiel entfernen
POST	/api/host/abort.php	Laufendes Spiel abbrechen

5.3 Inhalts-Endpoints (Dozent + Admin)

Diese Endpoints sind für Dozent und Admin gleichermaßen erreichbar. Sie betreffen Inhalte (Pools, Fragen, Bilder, Import) und Statistik.

Methode	URL	Zweck
GET / POST	/api/content/pools.php	Fragenpools auflisten, anlegen, ändern (ein Pool pro Thema)
GET / POST	/api/content/questions.php	Fragen auflisten, anlegen, ändern, deaktivieren
POST	/api/content/upload.php	Bild hochladen (multipart/form-data)
GET	/api/content/media.php	Vorhandene Bilder auflisten
POST	/api/content/import.php	Excel-/CSV-Datei in einen Pool importieren
GET	/api/stats/topics.php	Themen-Statistik: pro Pool/Frage Anzahl Versuche und Fehlerquote
POST	/api/content/archive-game.php	Altes Spiel archivieren

5.4 Statistik-Endpoint für eingeloggte User

Methode	URL	Zweck
GET	/api/stats/me.php	Eigene Historie: pro Thema Anzahl Spiele, Gesamtpunkte, Trefferquote

Aufrufbar von allen eingeloggten Nutzern (user, dozent, admin). Liefert ausschließlich Daten zum eigenen Account.

5.5 Verwaltungs-Endpoints (nur Admin)

Diese Endpoints sind exklusiv für die Rolle admin. Der Dozent hat hier keinen Zugriff – das ist eine bewusste Trennung.

Methode	URL	Zweck
GET	/api/admin/users.php	Alle User auflisten (E-Mail, Username, Rolle)
POST	/api/admin/users.php	Neuen User anlegen (mit Rolle user / dozent / admin)
PUT / POST	/api/admin/user-edit.php	User bearbeiten (Rolle ändern, Username ändern)
POST	/api/admin/user-delete.php	User löschen
POST	/api/admin/user-reset-password.php	Passwort zurücksetzen (Admin vergibt neues Passwort)

5.6 Beispiel: Polling-Response für /api/game/state.php

Das ist der wichtigste Endpoint. Frontend ruft ihn jede Sekunde auf. Die Response enthält alles, was die UI gerade anzeigen muss. Der Status entscheidet, was zu rendern ist.

```
{
  "ok": true,
```



```

"data": {
  "status": "running",           // lobby | running | reveal | between |
finished | aborted
  "current_index": 2,           // 0-basiert
  "total_questions": 10,
  "question": {
    "text": "Was macht der Befehl ls?",
    "image_url": null,
    "answers": [                // bereits gemischt, fixe Reihenfolge
      {"key": "x1", "text": "Listet Dateien auf"},
      {"key": "x2", "text": "Löscht Dateien"},
      {"key": "x3", "text": "Startet Linux"},
      {"key": "x4", "text": "Öffnet den Editor"}
    ]
  },
  "time_remaining_sec": 18,      // vom Server berechnet
  "committed_count": 7,         // wie viele haben eingeloggt
  "total_count": 12,
  "reveal": null,               // bei status='reveal': korrekte Antworten +
Erklärung
  "leaderboard": null           // bei status='reveal'/'finished': Rangliste
}
}

```

Wichtig: keys statt A/B/C/D nach außen

Wenn ihr nach außen die Buchstaben A,B,C,D verwendet, könnte ein Teilnehmer aus der Position auf die richtige Antwort schließen. Verwendet deshalb anonyme keys (x1..x4 oder UUIDs) im JSON für die laufende Frage. Erst beim Reveal kann das Backend zeigen, welche keys korrekt waren.

6. Ordnerstruktur des Projekts

Legt diese Struktur am Tag 1 an. Sie entscheidet, wer wo arbeitet und wer wessen Code anfasst.

```

quiz-webapp/
├── public/                                # Web-Root, hier zeigt Apache hin
│   ├── index.php                        # Startseite
│   ├── join.php                        # Beitreten als Gast / Login
│   ├── lobby.php                       # Teilnehmer-Lobby
│   ├── play.php                       # Teilnehmer spielt
│   ├── host.php                       # Host-Steuerung + Beameransicht
│   ├── result.php                     # Endranking
│   ├── login.php                      # Login für user/dozent/admin
│   ├── dashboard.php                 # User-Dashboard mit eigener Statistik
│   ├── content/                      # Inhalts-Verwaltung (Dozent + Admin)
│   │   ├── pools.php
│   │   ├── questions.php
│   │   ├── media.php
│   │   ├── import.php
│   │   └── topic-stats.php           # Themen-Übersicht für Dozent/Admin
│   ├── admin/                        # exklusive Admin-Verwaltung
│   │   ├── users.php
│   │   ├── user-edit.php
│   │   └── user-reset.php
│   ├── assets/
│   │   ├── css/style.css
│   │   ├── js/
│   │   │   ├── common.js
│   │   │   ├── play.js
│   │   │   ├── host.js
│   │   │   └── lobby.js
│   │   └── img/                     # Statische Bilder, Logos
│   ├── uploads/
│   │   └── questions/               # hochgeladene Frage-Bilder
│   └── api/                          # AJAX-Endpoints
│       ├── game/
│       ├── host/
│       ├── participant/
│       ├── content/                 # Pools, Fragen, Bilder, Import (Dozent + Admin)
│       ├── stats/                  # eigene Statistik + Themen-Übersicht
│       ├── admin/                  # Benutzerverwaltung (nur Admin)
│       └── auth/
└── src/                              # PHP-Klassen / Logik außerhalb Web-Root
    ├── Database.php                # PDO-Wrapper

```

```

|   ├── GameService.php      # Spiel-Logik (Erstellen, Starten, Bewerten)
|   ├── ScoreCalculator.php  # Punkteberechnung
|   ├── CsvImporter.php
|   ├── Auth.php
|   └── Security.php
|
|   ├── config/
|   │   ├── config.example.php # Vorlage mit Platzhaltern (in Git)
|   │   └── config.php         # echte Credentials (NICHT in Git)
|   │
|   ├── db/
|   │   ├── init.sql          # Schema-Anlage
|   │   ├── seed.sql          # Test-Daten: je 1 Admin/Dozent/User, 1 Pool, 10
|   │   └── Fragen
|   │       └── reset.sh       # Lokales Skript: drop + init + seed
|   │
|   ├── docs/
|   │   ├── api.md            # Endpoint-Doku
|   │   └── erd.png           # Datenbank-Diagramm
|   │
|   ├── tests/                # einfache manuelle Test-Skripte
|   ├── .gitignore
|   └── README.md

```

Sicherheit: config.php und uploads/

config.php enthält DB-Passwörter und gehört NICHT in Git. Trage sie sofort in .gitignore ein, sonst landen Credentials irgendwann öffentlich auf GitHub. Der Ordner uploads/questions/ braucht eine .htaccess, die PHP-Ausführung verbietet (php_flag engine off). Sonst können Angreifer eine getarnte PHP-Datei hochladen und ausführen.

7. Aufgabenteilung für 3 Personen

Diese Aufteilung folgt nicht Frontend/Backend, sondern Feature-Säulen. Das ist für 8 Tage praktischer, weil jeder seinen Bereich vertikal versteht – von der Datenbank bis zur UI.

7.1 Drei Säulen

Säule	Verantwortlich für	Lastenheft-Kapitel
A – Gameplay-Core	Spiel erstellen, Lobby, Frageablauf, Timer, Antworten, Punkte, Polling, Aggregations-Queries für Statistik	9, 10, 11, 12, 13, 14, 15–18, 20, 26
B – Spieler-Frontend, Auth & User-Statistik	Startseite, Beitreten, Spieler-UI, Login/Registrierung (3 Rollen), eigenes Dashboard mit Historie	7, 8, 22, 21
C – Inhalt, Statistik-UI & Benutzerverwaltung	Dozent-Bereich (Pools, Fragen, Bilder, Excel-/CSV-Import, Themen-Statistik-UI), Admin-Bereich (User-Verwaltung)	23, 24, 25, 27

7.2 Detaillierte Aufgaben pro Säule

Entwickler A – Gameplay-Core

Hat den schwierigsten Job, weil hier die Spiellogik sitzt. Sollte stark in PHP/SQL sein.

- Datenbank: games, game_participants, game_questions, participant_answers, game_results
- API-Endpoints: /api/host/*, /api/game/state.php, /api/participant/answer.php, /api/participant/commit.php
- Klasse GameService: Spiel erzeugen, join_code generieren, host_token generieren, Fragen zufällig ziehen, Antworten mischen
- Klasse ScoreCalculator: Punkte berechnen für beide Modi (Ganz oder gar nicht, Teilpunkte) inkl. Zeitbonus
- Server-Timer-Logik (current_question_started_at + time_limit_sec)
- Statusübergänge des Spiels: lobby → running → reveal → between → running → ... → finished
- Ranking-Berechnung inkl. korrektem Gleichstand (Lastenheft Kap. 20)
- Backend-Aggregations-Queries für Statistik: /api/stats/me.php (eigene Historie) und /api/stats/topics.php (Themen-Übersicht für Dozent/Admin)

Entwickler B – Spieler-Frontend, Auth & User-Statistik

Macht das, was die Teilnehmer am häufigsten sehen. Solide HTML/CSS/JS-Kenntnisse wichtig.

- public/index.php – Startseite mit Teilnehmen / Hosten / Login
- public/join.php – Code + Nickname + Avatar-Auswahl
- public/lobby.php – Wartebereich Teilnehmer
- public/play.php – Frageansicht: Frage, Antworten, Timer, Speichern-Button
- public/host.php – Host-/Beameransicht

- public/result.php – Endranking
- public/login.php + public/register.php – Auth-Formulare; nach Login Weiterleitung je nach Rolle
- public/dashboard.php – User-Dashboard mit eigener Statistik pro Thema (für user, dozent, admin)
- assets/css/style.css – Layout: 2x2-Raster für Antworten am PC/Laptop, klare Lesbarkeit auch am Beamer (Lastenheft Kap. 12.2)
- assets/js/play.js – Polling, Antwort-Auswahl, Timer-Anzeige, Antwort an /api/participant/answer.php senden
- assets/js/host.js – Host-Steuerung: Start, Next, Kick
- Rollen-bewusste Navigation: Menü zeigt nur, was die aktuelle Rolle darf

Entwickler C – Inhalt, Statistik-UI & Benutzerverwaltung

Bereitet die Daten vor, ohne die das Spiel keinen Inhalt hat. Muss früh liefern, damit A und B testen können. Hat zwei Hüte: Dozent-Bereich (für Dozent + Admin sichtbar) und exklusiver Admin-Bereich (Benutzerverwaltung).

Datenbank und gemeinsame Grundlagen:

- Datenbank: users, question_pools, questions, media + Seed-Daten anlegen (init.sql, seed.sql)
- Klasse Auth: Login, Logout, Rollen-Check (user / dozent / admin), password_hash() + password_verify()
- Klasse Security: Input-Validierung, XSS-Escape-Helper, CSRF-Token
- Helper-Funktion requireRole(['dozent', 'admin']) für Endpoint-Schutz

Dozent-Bereich (zugänglich für dozent und admin):

- public/content/pools.php – Liste aller Themen-Pools, Anlegen, Bearbeiten (ein Pool pro Thema)
- public/content/questions.php – Fragen-CRUD mit Validierung (1-4 richtige Antworten, genau 4 Antworten)
- public/content/media.php – Bild-Upload mit Sicherheitsprüfung (Dateityp, -größe, MIME), Bibliothek-Ansicht
- public/content/import.php – CSV-Import (Excel-Export → CSV) mit Zeilenvalidierung und Komplet-Abbruch bei Fehler
- public/content/topic-stats.php – Themen-Statistik-Dashboard: pro Pool und Frage die Fehlerquote (Daten von Entwickler A via /api/stats/topics.php)
- Klasse CsvImporter: parsen, validieren, Fehlerliste sammeln, alles oder nichts (Transaktion)

Admin-Bereich (exklusiv für admin):

- public/admin/users.php – Liste aller User, Anlegen neuer User, Rollenzuweisung (user / dozent / admin)
- public/admin/user-edit.php – User bearbeiten: Username, Rolle ändern
- public/admin/user-reset.php – Passwort zurücksetzen (Admin vergibt neues, User wird informiert)

- Löschen mit Sicherheitsabfrage; Schutz: Admin kann sich nicht selbst löschen, letzter Admin darf nicht gelöscht werden

Sicherheit (gemeinsam für beide Bereiche):

- .htaccess in uploads/questions/ zur Verhinderung von PHP-Ausführung
- Jeder Inhalts-Endpoint prüft: Rolle ist dozent oder admin
- Jeder Admin-Endpoint prüft: Rolle ist exakt admin

7.3 Gemeinsame Aufgaben (alle drei)

- Tag 1: Datenbank- und API-Entwurf
- Daily Standups + Integration jeden Nachmittag
- Tag 7-8: Gemeinsames Testing, Bugfixing, Abnahme-Durchlauf, Doku
- README schreiben mit Setup-Anleitung

Schnittstellen-Verträge

A liefert die API. B konsumiert sie. Damit B nicht warten muss, bis A fertig ist, definiert ihr am Tag 1 die JSON-Responses verbindlich. B kann dann mit Mock-Daten arbeiten (ein hartcodiertes JSON-Beispiel in einer Test-Datei), bis As echte Implementierung steht. Genauso konsumiert A am Ende die Daten, die C in die DB importiert hat.

8. Der 8-Tage-Plan

Das ist ein realistischer Plan. Er hat Puffer am Ende, weil Bugs immer am letzten Tag auftauchen. Wenn ihr Tag 1 mit Code statt Planung verbringt, schiebt sich alles um 1-2 Tage nach hinten und ihr werdet nicht fertig.

Tag 1 – Planung und Setup

Zeit	Was
Vormittag	Lastenheft gemeinsam durchgehen, Datenbankmodell entwerfen, API definieren, UI skizzieren, Aufgaben verteilen
Nachmittag	Repository und Ordnerstruktur anlegen, lokales Setup bei allen, erste init.sql mit allen Tabellen, Aufgaben für jeden definieren
Abend / Ende	Alle drei haben ihre lokale Umgebung am Laufen, ein erster Commit aus jeder Säule liegt in dev

Tag 2 – Fundamente

Entwickler	Ziel
A	Datenbank-Connection (Database.php), Skeleton GameService, Endpoint /api/host/create.php erzeugt einen Eintrag in games mit Code und Token
B	Startseite, statisches join.php-Formular, statisches play.php mit hartcodierter Beispiel-Frage in 2x2-Layout
C	seed.sql mit 1 Admin + 1 Dozent + 1 User und einem Demo-Pool, login.php funktioniert für alle drei Rollen, Auth-Klasse mit password_hash() und requireRole()-Helper

Tag 3 – Erste Verbindungen

Entwickler	Ziel
A	Endpoint /api/game/join.php (Gast tritt bei), /api/game/state.php gibt Lobby-Status zurück
B	join.php ruft echtes /api/game/join.php auf, Lobby pollt jede Sekunde state.php und zeigt Teilnehmerliste
C	content/pools.php (Liste + Anlegen) für Dozent und Admin sichtbar, content/questions.php als rohes Formular ohne Bild

Erste Integration: Eine Person erstellt ein Spiel (A), eine andere tritt bei (B), Admin hat im Pool die Demo-Fragen liegen (C). Das ist der erste Moment, in dem alle drei Säulen zusammenspielen. Wenn das funktioniert, ist die größte Hürde geschafft.

Tag 4 – Spielablauf

Entwickler	Ziel
A	/api/host/start.php zieht Fragen zufällig, mischt Antworten, persistiert in game_questions. state.php liefert die aktive Frage. /api/host/next.php schaltet weiter.
B	play.php zeigt echte Frage aus state.php, Auswahl wird an /api/participant/answer.php geschickt, Speichern-Button löst commit aus
C	content/questions.php komplett: Erstellen, Bearbeiten, Validierung (1-4 richtige Antworten), Deaktivieren. Erste Version Bild-Upload.

Wochenmitte-Review am Nachmittag: Was funktioniert? Was wackelt? Welche SOLL-Features kürzen wir, wenn es eng wird?

Tag 5 – Bewertung und Logik

Entwickler	Ziel
A	ScoreCalculator: beide Modi, Zeitbonus, runden. Reveal-Phase: Lösung im state.php-Response, Zwischenstand
B	Reveal-Anzeige (richtige Antworten farbig markiert, optional Erklärung), Zwischenstand-Tabelle, Antwort-Buttons groß und beamer-tauglich gestalten
C	CSV-Import: Parser, Validierung, alles-oder-nichts-Transaktion, Fehlerliste anzeigen. Medienbibliothek mit Vorschaubildern.

Tag 6 – Statistiken und Benutzerverwaltung

Entwickler	Ziel
A	Spielende: game_results schreiben (nur für registrierte User), Ranking mit Gleichstand-Logik, /api/host/abort.php, /api/host/kick.php, Aggregations-Queries für /api/stats/me.php und /api/stats/topics.php
B	result.php mit Endranking, dashboard.php für eingeloggte User mit eigener Themen-Historie, Rollen-bewusste Navigation
C	Themen-Statistik-UI für Dozent/Admin (content/topic-stats.php), Benutzerverwaltung (admin/users.php mit Anlegen, Bearbeiten, Löschen, Passwort zurücksetzen), Spielarchivierung, Sicherheits-Review (alle Inputs validiert? alle Outputs escaped? alle Prepared Statements? .htaccess in uploads/?)

Tag 7 – Integration und Bugfixing

Kein neuer Code mehr außer Bugfixes. Heute geht es nur darum: Funktioniert ein kompletter Durchlauf laut Lastenheft Kap. 34?

- Vormittag: Alle drei sitzen zusammen und spielen das System wie Endnutzer. Einer ist Admin, einer Host, einer Teilnehmer. Bugs werden in eine Liste geschrieben.
- Nachmittag: Bugs nach Schwere abarbeiten. MUSS-Features zuerst. Bei Zeit: ausgewählte SOLL-Features.

- Abend: Test mit mehreren echten PCs/Laptops im LAN, falls möglich. Mindestens aber 3-5 parallele Browser-Fenster (verschiedene Profile oder Inkognito) als Teilnehmer-Simulation.

Tag 8 – Polish, Test, Präsentation

- Vormittag: Alle Testfälle aus Lastenheft Kap. 32 durchgehen, abhaken oder fixen.
- Nachmittag: README finalisieren mit Setup-Anweisungen, Screenshots ergänzen, ggf. kurzes Demo-Video drehen.
- Generalprobe für Präsentation/Abnahme: Kompletter Durchlauf laut Lastenheft Kap. 34, gemeinsam, mit echten Geräten.

Goldene Regel: Tag 8 ist kein Coding-Tag

Wenn ihr an Tag 8 noch Features schreibt, habt ihr verloren. Am Tag 8 wird nur noch poliert, getestet und vorbereitet. Wer das nicht einhält, gibt am Abend ein Projekt ab, in dem ein neues Feature drin ist – und drei alte kaputt sind, weil zum Testen keine Zeit blieb.

9. In welcher Reihenfolge wird gebaut?

Innerhalb jeder Säule gibt es eine sinnvolle Reihenfolge. Diese Reihenfolge folgt dem Spielablauf, nicht der Komplexität. Das heißt: Ihr baut Schritt 1 des Spiels (Erstellen), dann Schritt 2 (Beitreten), dann Schritt 3 (Starten) usw. So habt ihr früh etwas Spielbares.

9.1 Vertikale Slices

Statt erst alle Modelle, dann alle APIs, dann alle UIs zu bauen, baut ihr pro Feature einen kompletten vertikalen Slice durch alle Schichten.

- 22.Slice 1: Spiel erstellen → DB-Tabelle games, Endpoint create, Host-UI mit Code-Anzeige. Ende: Host sieht einen Code.
- 23.Slice 2: Spiel beitreten → Endpoint join, Teilnehmer-Form, Lobby. Ende: Mehrere Browser sehen sich gegenseitig in der Lobby.
- 24.Slice 3: Spiel starten → Fragen ziehen, mischen, persistieren. Ende: Erste Frage erscheint überall.
- 25.Slice 4: Antwort speichern → Auswahl-UI, answer + commit Endpoints. Ende: Teilnehmer kann antworten, Host sieht den Counter steigen.
- 26.Slice 5: Frage abschließen → Server-Timer, Reveal, Zwischenstand. Ende: Lösung erscheint, Punkte werden gezeigt.
- 27.Slice 6: Nächste Frage → Statusübergang. Ende: Durchlauf bis zur letzten Frage.
- 28.Slice 7: Endranking → game_results, result.php. Ende: Spiel ist komplett spielbar.
- 29.Slice 8: Admin → Fragenverwaltung, CSV, Upload. Ende: Eigene Inhalte können gepflegt werden.

Slice 1-7 ergeben das spielbare Minimum. Mit Slice 8 + Härtung + Polish habt ihr die Abnahme.

9.2 Was wird zuerst getan?

Am Tag 1 nachmittags, in dieser Reihenfolge:

- 30.Ordnnerstruktur anlegen (5 Min)
- 31.init.sql schreiben und Datenbank befüllen (45 Min)
- 32.config.php und Database.php (PDO mit Prepared Statements) – das ist die Brücke zur DB, ohne sie geht nichts (45 Min)
- 33.seed.sql mit Admin-User + 1 Demo-Pool + 10 Test-Fragen, damit alle gleich beginnen können (30 Min)
- 34.Ein kleines „Hello World“ in PHP, das aus der DB liest und auf einer Seite anzeigt – Beweis, dass das Setup steht (15 Min)

Erst danach beginnt jeder mit seinem ersten Slice.

10. Technische Entscheidungen und Stolperfallen

10.1 Server-Timer korrekt implementieren

Das Lastenheft (Kap. 13.4 und 26.3) ist hier klar: Der Server entscheidet, wie viel Zeit übrig ist. Der Browser zeigt nur an.

Falsche Lösung: JavaScript-Timer im Browser, der nach Ablauf eine API-Call macht. Problem: Browser-Uhren weichen ab, Tab-Wechsel friert den Timer ein, manipulierbar durch geöffnete DevTools.

Richtige Lösung:

- Beim Frageschritt: `games.current_question_started_at = NOW()` setzen
- Bei `state.php`: `time_remaining_sec = time_limit_sec - (NOW() - current_question_started_at)`
- Bei `answer.php` / `commit.php`: Server prüft, ob die Restzeit noch ≥ 0 ist. Wenn nein: 0 Punkte
- Das Frontend zeigt `time_remaining_sec` aus dem Polling-Response an, zählt aber nicht selbst herunter zwischen den Polls (oder optional client-seitig dekrementieren, aber der Server hat das letzte Wort)

10.2 Polling-Strategie

Empfehlung des Lastenhefts: 1 Sekunde Polling-Intervall. Das ist okay für Klassen mit bis zu 30 Teilnehmern. Tipps:

- Polling pausieren, wenn der Browser-Tab nicht aktiv ist (`document.hidden`), um Last zu reduzieren
- Eine ETag- oder Hash-Logik im `state.php`, sodass der Client weiß, ob sich etwas geändert hat – spart UI-Re-Renders
- Lobby-Polling alle 2 Sekunden reicht, im laufenden Spiel 1 Sekunde

10.3 join_code und host_token sauber generieren

Verwechslungsanfällige Zeichen vermeiden (Lastenheft Kap. 8.3):

```
$alphabet = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789'; // ohne I, O, 0, 1
function generateJoinCode(int $len = 5): string {
    global $alphabet;
    $code = '';
    for ($i = 0; $i < $len; $i++) {
        $code .= $alphabet[random_int(0, strlen($alphabet) - 1)];
    }
    return $code;
}
// host_token:
$hostToken = bin2hex(random_bytes(32)); // 64 Zeichen, kryptographisch sicher
```

Bei Code-Generierung in einer Schleife auf Eindeutigkeit prüfen (UNIQUE-Constraint reicht eigentlich, aber höflich ist es trotzdem):

```
do { $code = generateJoinCode(); } while (gameWithCodeExists($code));
```

10.4 Punkte korrekt berechnen

Hier ein Pseudocode für die kritische Stelle. Achtet darauf: Das Lastenheft unterscheidet zwischen vollständig richtig (zählt in `fully_correct_count`) und teilrichtig (zählt nicht, aber gibt halbe Punkte).

```
function calculatePoints(
    array $correctSet,    // ['A','C'] (Antworten, die korrekt sind)
    array $selectedSet,  // ['A','C'] (was der Teilnehmer gewählt hat)
    int $timeLimitSec,
    int $timeRemainingSec,
    string $scoreMode    // 'all_or_nothing' | 'partial'
): array {
    $isFullyCorrect = count(array_diff($correctSet, $selectedSet)) === 0
        && count(array_diff($selectedSet, $correctSet)) === 0;

    $hasFalseSelected = count(array_diff($selectedSet, $correctSet)) > 0;
    $hasCorrectSelected = count(array_intersect($selectedSet, $correctSet)) > 0;

    $timePoints = (int) round(
        500 + 500 * ($timeRemainingSec / $timeLimitSec)
    );

    if ($isFullyCorrect) {
        return ['points' => $timePoints, 'fully_correct' => true];
    }

    if ($scoreMode === 'partial'
        && $hasCorrectSelected
        && !$hasFalseSelected) {
        return ['points' => (int) round($timePoints / 2), 'fully_correct' =>
false];
    }

    return ['points' => 0, 'fully_correct' => false];
}
```

10.5 Ranking mit Gleichstand

Das Lastenheft (Kap. 20) verlangt: Bei gleicher Punktzahl gleicher Platz, der nächste Platz wird übersprungen (DENSE_RANK wäre falsch, RANK richtig). Beispiel: 1, 1, 3, 4.

```
SELECT participant_id, total_points,
```

```
RANK() OVER (ORDER BY total_points DESC) AS rank_position
FROM (...)
```

Wenn ihr MariaDB < 10.2 verwendet, gibt es kein RANK(). Dann macht ihr es in PHP:

```
$rank = 0;
$prevPoints = null;
$rowsProcessed = 0;
foreach ($sortedByPointsDesc as $row) {
    $rowsProcessed++;
    if ($row['total_points'] !== $prevPoints) {
        $rank = $rowsProcessed;
        $prevPoints = $row['total_points'];
    }
    $row['rank_position'] = $rank;
}
```

10.6 Polling-Last bei vielen Teilnehmern

Bei 25 Teilnehmern und 1-Sekunden-Polling sind das 25 DB-Hits pro Sekunde nur für state.php. Das ist okay, solange die Query schnell ist. Optimierungen:

- Indizes auf games.join_code, game_participants.game_id, participant_answers.game_question_id
- state.php liest nicht alle Daten, sondern nur was nötig ist
- Im Cache (z. B. eine einfache file_cache.php) den state für ca. 500 ms zwischenspeichern, falls Performance Probleme macht

Was ihr NICHT braucht

Keine Frameworks (Laravel, Symfony) – das Lastenheft verbietet es ausdrücklich. Keine WebSockets – Polling reicht. Kein npm/Composer-Wildwuchs. Kein TypeScript. Kein Webpack. Kein Tailwind über CDN ist okay, aber Vanilla CSS ist sauberer. Bleibt einfach – das ist im Unterricht der ganze Punkt.

11. Sicherheits-Checkliste

Bewertet wird auch eure Sicherheit (Lastenheft Kap. 33). Diese Checkliste arbeitet ihr spätestens an Tag 6 systematisch durch.

11.1 SQL-Injection verhindern

- Alle DB-Queries laufen über PDO mit Prepared Statements und gebundenen Parametern
- Niemals Strings in Queries konkatenieren, auch nicht bei „nur Zahlen“

```
// FALSCH
$pdo->query("SELECT * FROM users WHERE email = '$email'");

// RICHTIG
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = ?');
$stmt->execute([$email]);
```

11.2 XSS verhindern

- Jede Ausgabe von Nutzereingaben mit htmlspecialchars(\$wert, ENT_QUOTES, 'UTF-8') escapen
- Besonders kritisch: Nicknames, Fragentexte, Antworten, Erklärungen
- In JSON-Responses ist json_encode mit Standardoptionen sicher, aber im HTML-Kontext immer escapen

```
// In allen PHP-Templates
function e($s) { return htmlspecialchars($s, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8'); }
echo '<p>' . e($participant['display_name']) . '</p>';
```

11.3 Passwort-Hashing

- Nur password_hash(\$pw, PASSWORD_DEFAULT) verwenden
- Prüfen mit password_verify(\$pw, \$hash)
- Niemals MD5, SHA1 oder Klartext

11.4 Upload-Sicherheit

- Dateigröße prüfen (max 20 MB, Lastenheft Kap. 24.1)
- Erlaubte MIME-Types: image/jpeg, image/png, image/webp
- MIME-Type mit finfo_file() prüfen, nicht der Browser-Angabe vertrauen
- Dateiendung serverseitig neu setzen, nie den Originalnamen direkt verwenden
- Eindeutiger Dateiname per bin2hex(random_bytes(16)) . '.jpg'
- .htaccess in uploads/questions/ mit php_flag engine off (oder Apache: <FilesMatch "\.ph(p[0-9]?|tml)\$">Require all denied</FilesMatch>)

11.5 Host-Token und Admin-Auth

- Jeder Host-Endpoint prüft den Token aus dem Request gegen games.host_token (Lastenheft Kap. 27.5)
- Inhalts-Endpoints (/api/content/*) prüfen \$_SESSION['user_id'] und users.role IN ('dozent','admin')
- Admin-exklusive Endpoints (/api/admin/users.php usw.) prüfen \$_SESSION['user_id'] und users.role = 'admin' (Lastenheft Kap. 27.6)
- Host-Endpoints (/api/host/*) sind für Admins gesperrt – Rollen-Check verbietet das (eure neue Anforderung)
- Session-Cookies mit session_set_cookie_params(['httponly' => true, 'samesite' => 'Strict'])

Empfohlener Helper, der in jedem geschützten Endpoint ganz oben aufgerufen wird:

```
function requireRole(array $allowed): void {
    if (!isset($_SESSION['user_id'])) {
        http_response_code(401);
        echo json_encode(['ok' => false, 'error' => 'Nicht angemeldet']);
        exit;
    }
    if (!in_array($_SESSION['role'], $allowed, true)) {
        http_response_code(403);
        echo json_encode(['ok' => false, 'error' => 'Keine Berechtigung']);
        exit;
    }
}

// Anwendung am Anfang des Endpoints:
requireRole(['dozent', 'admin']); // Inhalts-Verwaltung
requireRole(['admin']);          // nur Benutzerverwaltung
requireRole(['user', 'dozent']); // Hosting (Admin gesperrt)
```

11.6 CSRF bei Form-POSTs im Admin

Optional aber empfohlen: CSRF-Token in Admin-Forms.

```
// Beim Login / Session-Start:
$_SESSION['csrf'] = bin2hex(random_bytes(32));

// In Forms:
<input type="hidden" name="csrf" value="<? e($_SESSION['csrf']) ?>">

// Beim Verarbeiten:
if (!hash_equals($_SESSION['csrf'], $_POST['csrf'] ?? '')) { die('Ungültig'); }
```

12. Test-Strategie

Automatisierte Tests sind in 8 Tagen Luxus. Wichtig ist ein strukturierter manueller Test, der alle Testfälle aus Lastenheft Kap. 32 abdeckt.

12.1 Testprotokoll vorbereiten

Legt eine einfache Tabelle (Excel oder Markdown) an. Eine Zeile pro Testfall aus dem Lastenheft. Spalten: ID, Beschreibung, Erwartetes Ergebnis, Status (offen / okay / fehlerhaft), Bemerkung.

12.2 Wichtige End-to-End-Tests

Nr.	Szenario
1	Host erstellt Spiel, 3 Teilnehmer treten bei, Host startet, alle 10 Fragen werden beantwortet, Endranking erscheint korrekt
2	Pool hat nur 5 Fragen, Host wählt 10 → Fehlermeldung, Spiel startet nicht
3	Teilnehmer wechselt während laufender Frage seine Antwort dreimal – die letzte gilt
4	Ein Teilnehmer beantwortet nichts, Timer läuft ab → 0 Punkte, gilt als falsch
5	Spiel mit 3 Teilnehmern, zwei haben gleiche Endpunkte → beide Platz 1, nächster Platz 3
6	Host entfernt einen Teilnehmer mitten im Spiel – dieser bekommt eine Meldung, kann nicht mehr antworten
7	CSV-Import mit 10 korrekten Zeilen → alle importiert. CSV mit Fehler in Zeile 3 → kompletter Abbruch, keine Frage importiert, Fehlerliste
8	Bild-Upload von 25 MB-Datei → abgelehnt. Bild als gefälschte .php-Datei → abgelehnt
9	Admin bearbeitet eine Frage so, dass keine Antwort als richtig markiert ist → Speichern verweigert
10	Frage in laufendem Spiel – Host sieht Counter wie 7/12 geantwortet, nicht aber welche Antworten
11	Test mit 5+ parallelen Browser-Fenstern als Teilnehmer: Alle sehen synchron die gleiche Frage, gleiche Antwortreihenfolge, gleicher Timer
12	Im Modus „Teilpunkte“: Antwort teilrichtig (kein Fehler) gibt halbe Punkte, mit Fehler 0 Punkte

12.3 Wer testet was?

Beim großen Test am Tag 7-8 wechselt ihr die Rollen. Wer A entwickelt hat, testet jetzt das Spielen aus Teilnehmer-Sicht. Wer C entwickelt hat, testet jetzt den Spielablauf. Fremde Augen finden mehr Bugs als die des Autors.

13. Mapping: Lastenheft-Anforderung zu Umsetzung

Diese Tabelle ist eure Versicherung gegen vergessene Anforderungen. Geht sie an Tag 8 noch einmal durch – jedes MUSS muss grün sein.

13.1 Muss-Funktionen aus Kap. 29

Anforderung	Umgesetzt in	Säule
Startseite mit Teilnehmern/Hosten	public/index.php	B
Gast kann Spiel hosten und beitreten	/api/host/create.php, /api/game/join.php	A
Teilnahme-Code und Host-Token werden erzeugt	GameService::createGame()	A
Host wählt Pool, Anzahl, Zeitlimit, Modus, Mitspielen	host.php Formular, /api/host/create.php	A+B
Teilnehmer Anzeigename, nur in Lobby beitreten	join.php, games.status='lobby' Check	B+A
Host sieht Teilnehmerliste, kann entfernen, Spiel starten	host.php, /api/host/kick.php, /api/host/start.php	A+B
Fragen zufällig gezogen, Antworten gemischt	GameService::startGame()	A
Frage groß angezeigt, optional Bild, 4 Antworten	play.php, host.php	B
Layout 2x2 für PC/Laptop	style.css	B
Antworten auswählen, speichern, bis Ende änderbar	play.js, /api/participant/answer.php	B+A
Frage endet wenn alle gespeichert oder Timer abläuft	state.php-Logik in GameService	A
Timer serverseitig	current_question_started_at + time_limit_sec	A
Lösung und Zwischenstand nach Frage	state.php status='reveal'	A+B
Host klickt Nächste Frage	/api/host/next.php	A+B
Endranking mit korrektem Gleichstand	result.php, RANK-Logik	A+B
Ergebnisse dauerhaft gespeichert	game_results-Tabelle	A
Admin-Login über Seed-Account	seed.sql, login.php	C
Dozent verwaltet Pools, Fragen, Bilder, Import; Admin darf das ebenfalls	/content/* + /api/content/*	C
Fehlerhafter CSV-Import bricht komplett ab, Fehlerliste	CsvImporter (Transaktion)	C
Spiele archivieren	/api/content/archive-game.php, status='archived'	C

Anforderung	Umgesetzt in	Säule
Admin verwaltet Benutzer (anlegen, bearbeiten, löschen, Passwort zurücksetzen)	/admin/users.php, /admin/user-*	C
User sieht eigene Statistik pro Thema	dashboard.php, /api/stats/me.php	A+B
Dozent und Admin sehen Themen-Übersicht (falsch beantwortete Fragen)	content/topic-stats.php, /api/stats/topics.php	A+C

13.2 Soll-Funktionen aus Kap. 30 (wenn Zeit)

Anforderung	Aufwand	Empfehlung
Erklärung nach jeder Frage	klein	Mitnehmen, ist 1 Stunde
Avatar pro Nutzer	klein	Auswahl aus 8 Symbolen reicht
Medienbibliothek mit Vorschau	mittel	Pflicht für Bildauswahl, also ohnehin in Säule C
Spielübersicht für Administratoren	mittel	Eigene Seite admin/games.php
Bessere optische Gestaltung	groß	Tag 7-8 als Polish
Saubere Beamer-Darstellung (große Schrift, Kontrast)	klein	Wichtig für Vorführung, in Säule B
QR-Code für Teilnahme-Code	klein	Ohne WLAN nutzlos – streichen
Echter XLSX-Import (statt CSV)	mittel	Nur wenn Tag 7 fertig

Pragmatische Empfehlung

Plant SOLL-Features bewusst NICHT für die ersten 5 Tage ein. Wenn sie als Bonus auftauchen, sind alle glücklich. Wenn ihr sie früh einbaut und es eng wird, müsst ihr MUSS-Features streichen oder unfertig abgeben. Das wäre ein vermeidbarer Bewertungsverlust.

14. Abnahme-Checkliste

Vor der Abnahme arbeitet ihr diese Liste gemeinsam durch. Jeder Punkt entspricht Kap. 34 des Lastenhefts.

1. Dozent (oder Admin) loggt sich ein und hat einen Fragenpool mit mind. 10 aktiven Fragen (per Excel-/CSV-Import oder manuell) angelegt
2. Host (anderer Browser) erstellt Spiel aus diesem Pool
3. Teilnahme-Code wird groß und gut lesbar angezeigt
4. Mind. 3 Teilnehmer treten von verschiedenen PCs/Laptops bei (oder ersatzweise von verschiedenen Browser-Fenstern auf demselben Rechner)
5. Host startet das Spiel
6. Alle Fragen werden nacheinander gespielt, jeweils mit Timer und Speicher-Funktion
7. Antworten werden korrekt ausgewertet (Stichproben prüfen: vollständig, teilrichtig, falsch)
8. Lösung erscheint nach jeder Frage
9. Zwischenstand erscheint mit den richtigen Punkten
10. Host klickt jedes Mal aktiv auf Nächste Frage
11. Endranking erscheint mit korrekter Sortierung und Gleichstand-Logik
12. Ergebnis ist in der Datenbank gespeichert (game_results)
13. Anwendung funktioniert stabil auf PC/Laptop in den gängigen Browsern (Chrome und Firefox als Minimum)
14. Alle Eingaben werden serverseitig validiert (kurz testen: leerer Nickname, ungültiger Code)
15. CSV-Import: 1x mit fehlerhafter Datei → Abbruch, 1x mit korrekter Datei → erfolgreich
16. Bild-Upload: 1x mit zu großer Datei → Ablehnung, 1x normal → erfolgreich, Bild in Frage einbindbar

14.1 Zusätzliche Tests für das erweiterte Rollensystem

17. Gast spielt durch und sieht KEIN Dashboard / KEINE Statistik nach dem Spiel
18. User (registriert) spielt durch und sieht im Dashboard die Spielergebnisse pro Thema
19. Dozent öffnet Themen-Statistik und sieht aggregiert, welche Fragen am häufigsten falsch beantwortet wurden
20. Admin sieht die gleiche Themen-Statistik wie der Dozent
21. Admin kann einen neuen User mit jeder Rolle (user / dozent / admin) anlegen
22. Admin kann einen bestehenden User bearbeiten (Rollenwechsel von user → dozent funktioniert)
23. Admin kann das Passwort eines Users zurücksetzen, der neue User kann sich mit dem neuen Passwort einloggen
24. Admin kann einen User löschen, dieser kann sich danach nicht mehr einloggen
25. Admin versucht zu hosten oder beizutreten → wird abgewiesen (Rollen-Trennung greift)

- 26. Dozent versucht, einen anderen User zu löschen → wird abgewiesen (User-Mgmt nur für Admin)
- 27. User versucht, Themen-Statistik aufzurufen → wird abgewiesen (Statistik nur für Dozent/Admin)

14.2 Sauberer Übergabe-Stand

- README.md mit Setup-Anleitung (Datenbank importieren, Server starten, Admin-Login-Daten)
- seed.sql legt mindestens je einen Test-Account pro Rolle an: ein admin, ein dozent, ein user – mit dokumentierten Zugangsdaten in der README
- init.sql und seed.sql funktionieren auf einer frischen DB
- Keine Dummy-Credentials in Git, config.example.php als Vorlage
- Alle bekannten Bugs in einer KNOWN_ISSUES.md, falls etwas nicht funktioniert
- Demo-Daten: mind. 1 Pool mit 15+ Fragen, damit eine Live-Demo problemlos läuft

14.3 Präsentation der Anwendung

Plant für die Präsentation 10-15 Minuten ein:

- 1 Min: Was ist das Projekt, wer hat was gemacht?
- 2 Min: Architektur-Skizze (Frontend / PHP-API / DB / Polling)
- 8 Min: Live-Durchlauf eines kompletten Spiels (Host-PC am Beamer + 1-2 Teilnehmer-Laptops oder zusätzliche Browser-Fenster)
- 2 Min: Kurzer Blick in Admin-Bereich (CSV-Import, Frage anlegen mit Bild)
- Restzeit: Was würden wir mit mehr Zeit machen? (KÜR-Features, Verbesserungen)

Vor der Präsentation: Generalprobe

Macht spätestens am Vortag eine komplette Generalprobe in der echten Umgebung. Lokales Kursraum-Netzwerk, Beamer, mehrere PCs. Was auf einem Rechner mit mehreren Browser-Fenstern funktioniert, läuft nicht garantiert auf mehreren vernetzten PCs. Stichworte: Statische IP des Server-PCs, lokale Firewall-Einstellungen, Port-Freigaben, korrekte URL für die Teilnehmer-Rechner. Lieber einmal bei der Probe Probleme finden als bei der Abnahme.

15. Zusammenfassung – Die wichtigsten 10 Punkte

- 28. Investiert Tag 1 in Planung, nicht in Code. Datenbank und API zuerst.
- 29. Teilt nach Feature-Säulen auf (Gameplay / Spieler+Auth / Admin+Daten), nicht nach Schichten.
- 30. Baut in vertikalen Slices – jeder Slice bringt das System einen sichtbaren Schritt voran.
- 31. Der Server entscheidet alles: Timer, Antwortgültigkeit, Punkte. Browser zeigen nur an.
- 32. Antwortreihenfolge wird einmal beim Spielstart gemischt und persistiert – nie neu mischen.
- 33. Polling alle 1 Sekunde, ein einziger state.php-Endpoint liefert alles Notwendige.
- 34. MUSS vor SOLL vor KÜR. Streicht SOLL-Features ohne schlechtes Gewissen, wenn es eng wird.
- 35. Sicherheit ist Pflicht, nicht Kür: Prepared Statements, htmlspecialchars, password_hash, Upload-Härtung.
- 36. Tag 7 und 8 sind keine Coding-Tage. Sie sind für Integration, Tests und Polish.
- 37. Macht eine Generalprobe in der echten Umgebung, bevor ihr abnehmt.

Viel Erfolg!

Wenn ihr diesen Leitfaden Schritt für Schritt befolgt, bekommt ihr ein lauffähiges Projekt.